

DA2304: Introduction to Computer Systems

Assignment 3: Jack Compiler & 2D Convolution

Racharla Dhnaush-DA24B019

Part 1: The Jack Convolution Program

Exercise 1: 2D Convolution — Background

A 2D convolution slides a 3×3 kernel across a larger input matrix. At each position, the element-wise product of the kernel and the overlapping window is summed into a single output value. With no padding ($P = 0$) and filter size $F = 3$, the output dimension is:

$$O = \left\lfloor \frac{N - 3}{S} \right\rfloor + 1$$

For example, $N = 5$, $S = 1$ gives $O = 3$, and $N = 9$, $S = 2$ gives $O = 4$.

Exercise 2: The Conv Class

I used the Laplacian edge-detector:

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

This computes the discrete second derivative. On a linearly-varying input it always gives zero ($\nabla^2(ax + by + c) = 0$), making it a clean test case while still being a meaningful edge-detection filter.

Key design choices:

- `Conv.new(N)` sets `stride = 1`; `Conv.newWithStride(N, s)` allows `stride = 2` for the extra-credit test.
- All matrices are stored row-major. `ArrayInput` and `ArrayOutput` are field arrays; `ArrayFilter` and `stride` are static (shared across instances).
- If $N < 3$, `conv2d()` writes `-1` to `RAM[16384]` as an error flag.

Output dimension. Jack's `/` is integer (floor) division, so the formula `((ArraySize - 3) / stride) + 1` directly implements O with no extra helper.

Part 2: The Jack Compiler Frontend

Exercise 1: Tokenizer (JackTokenizer.py)

The tokenizer processes source code in two passes:

1. **Comment stripping:** A character-level state machine walks the source tracking three states: `normal`, `in_line_comment`, and `in_block_comment`. This correctly handles multi-line block comments and avoids stripping `/*` sequences that appear inside string literals.

2. **Token scanning:** The cleaned source is scanned left-to-right and each token is classified into one of five types: **keyword** (21 reserved words), **symbol** (19 punctuation characters), **integerConstant** (range 0–32767), **stringConstant** (double-quoted text), or **identifier** (letter/underscore-led names).

The output is written to `<ClassName>T.xml` with the five XML-unsafe characters (`<`, `>`, `&`, `"`, `'`) properly escaped.

Exercise 2: Parser & VM Code Generator

`CompilationEngine.py` is a recursive-descent parser that walks the Jack grammar and simultaneously outputs the parse-tree XML and VM code. Each grammar rule (e.g. `compileClass`, `compileSubroutine`) is one method; they call each other recursively and bottom out at `compileTerm`.

The `SymbolTable` uses two dictionaries — one for class scope (`static`, `field`) and one for subroutine scope (`argument`, `var`). Each entry stores (`type`, `kind`, `index`) and maps to a VM segment: `field` $\rightarrow$ `this`, `var` $\rightarrow$ `local`, `argument` $\rightarrow$ `argument`, `static` $\rightarrow$ `static`. The subroutine table is cleared at the start of every new subroutine.

Key VM generation rules: constructors call `Memory.alloc` and set `pointer 0`; methods load `argument 0` into `pointer 0` to anchor `THIS`; array writes use the `pointer 1/that 0` idiom; void return values are discarded with `pop temp 0`; and `true` is emitted as `push constant 0 / not`.

Exercise 3: Analysis Questions

Q1–Tokenizer design

A simple single-line regex like `re.sub(r'\/\*.*?\*\/', '', line)` fails whenever a block comment spans multiple lines. For example:

```
/* This comment
   spans three
   lines */
let x = 5;
```

A per-line regex would leave `spans three` and `lines */` in the token stream, causing the tokenizer to misclassify them as identifiers and produce a malformed output.

A second problem: the string `/* not a comment */` contains comment delimiters that must not be stripped. A regex applied before string detection would silently corrupt the string.

The state machine fixes both issues by tracking three states simultaneously — `normal`, `in_block_comment`, `in_string` — and handling all interactions correctly.

Q2 – Symbol table

The relevant line is:

```
let sum = sum + (ArrayInput[inputIdx] * ArrayFilter[filterIdx]);
```

Class scope:

Table 1: Class Scope Symbol Table

Name	Type	Kind	VM Segment	Index
ArrayFilter	Array	static	static	0
stride	int	static	static	1
ArrayInput	Array	field	this	0
ArrayOutput	Array	field	this	1
ArraySize	int	field	this	2

Subroutine of conv2d:

Table 2: Subroutine Scope Symbol Table for conv2d

Name	Type	Kind	VM Segment	Index
this	Conv	argument	argument	0
outDim	int	local	local	0
outRow	int	local	local	1
outCol	int	local	local	2
fRow	int	local	local	3
fCol	int	local	local	4
inRow	int	local	local	5
inCol	int	local	local	6
sum	int	local	local	7
k	int	local	local	8
inputIdx	int	local	local	9
filterIdx	int	local	local	10

Q3 –Compilation correctness

ArrayOutput is a field array, so `this[1]` holds the base address of the heap-allocated array. The pointer `1 / that 0` idiom is needed to dereference it:

```

1 push this 1      // base address of ArrayOutput
2 push local 8     // index k
3 add              // base + k = &ArrayOutput[k]
4 push local 7     // value of sum (RHS)
5 pop temp 0       // save sum before pointer is overwritten
6 pop pointer 1    // THAT = &ArrayOutput[k]
7 push temp 0      // restore sum
8 pop that 0       // ArrayOutput[k] = sum

```

The indirection is necessary because Jack arrays are heap objects and the variable only stores a base pointer. Setting `pointer 1` makes `that 0` resolve to the exact RAM cell of element k .

Q4 – Convolution stride

Output dimension for $N = 9$, $F = 3$, $S = 2$:

$$O = \left\lfloor \frac{9 - 3}{2} \right\rfloor + 1 = 3 + 1 = 4$$

The three nested loops in `conv2d()` iterate $O^2 \times 9$ times. Let $O = \lfloor (N - 3)/S \rfloor + 1$:

- $S = 1$: $O \approx N$, so iterations $\approx 9N^2 \Rightarrow \Theta(N^2)$
- $S = 2$: $O \approx N/2$, so iterations $\approx 9N^2/4 \Rightarrow \Theta(N^2)$

The asymptotic complexity is $\Theta(N^2)$ in both cases. Stride 2 reduces the constant factor by roughly $4\times$ but does not change the order of growth. Since the number of VM instructions per inner iteration is fixed (around 30), total instruction count scales as $\Theta(N^2)$ regardless of stride.

Extra Credit: Most Interesting Bug

The trickiest problem occurred when testing the programs with multiple `do` instructions invoking void subroutines. Initially the program performed as expected, but eventually the results became incorrect and in some cases the program crashed. After analyzing the output VM code, I noticed that in Jack all subroutines — whether void or not — pushed a dummy 0 value onto the stack at the time of return regardless of the type of the subroutine's output.

In my compiler, I only produced the `call` instruction without ever producing `pop temp 0`, which is responsible for removing the dummy return value from the stack. The consequence of this omission was that with every new `do` instruction the stack pointer incremented and the following `push local` and `push argument` instructions were accessing invalid memory cells. To solve this problem, I made the compiler produce `pop temp 0` after every invocation of a `do` statement. It was easy to verify this problem by watching the stack pointer in the CPU emulator increase by one after each `do` statement.